* Alguns dos Exercícios foram retirados/adaptados do curso Python Programming MOOC 2026

1) Pets (Animais de Estimação):

O modelo de exercício contém os esboços de duas classes: Person e Pet. Cada pessoa tem um animal de estimação.

Por favor, altere o método __str__ na classe Person para que ele também exiba informações sobre o animal de estimação da pessoa, conforme mostrado no exemplo abaixo.

Exemplo de Uso:

```
hulda = Pet("Hulda", "mixed breed")
hesi = Person("Hesi", hulda)

print(hesi)
```

Saída esperada:

```
Hesi, his pet is Hulda, a mixed breed
```

2) A box of presents (Uma caixa de presentes)

Observação: Alguns exercícios possuem múltiplas partes.

Neste exercício, você praticará a embalagem de presentes. Você escreverá duas classes: Present e Box. Um presente possui um nome e um peso, e uma caixa contém presentes.

A classe Present :

Por favor, defina a classe Present que pode ser usada para representar diferentes tipos de presentes. A definição da classe deve conter atributos para o nome (name) e o peso (weight em kg) do presente.

Instâncias da classe devem funcionar da seguinte maneira:

```
book = Present("ABC Book", 2)
print("The name of the present:", book.name)
print("The weight of the present:", book.weight)

print("Present:", book)
```

Saída esperada:

```
The name of the present: ABC Book
The weight of the present: 2
Present: ABC Book (2 kg)
```

A classe Box

Por favor, defina a classe Box. Você deve ser capaz de adicionar presentes à caixa, e a caixa deve manter o controle do peso combinado dos presentes contidos nela.

A definição da classe deve conter estes métodos:

- add_present(self, present: Present): adiciona o presente fornecido como argumento à caixa. O método não possui valor de retorno.
- total_weight(self): retorna o peso combinado dos presentes na caixa.

```
book = Present("ABC Book", 2)

box = Box()
box.add_present(book)
print(box.total_weight())

cd = Present("Pink Floyd: Dark Side of the Moon", 1)
box.add_present(cd)
print(box.total_weight())
```

Saída esperada:

```
2
3
```

3) Você está desenvolvendo uma aplicação de agenda de contatos. Crie duas classes: Contato e Agenda.

- A classe contato deve, minimamente, guardar um nome e um telefone (não públicos)
- A classe agenda deve conter uma lista de contatos
  - Métodos: Construtor, adicionar contato, remover contato, exibir contatos
  - Ao adicionar um contato a agenda deve ser organizada em ordem alfabética.
- Elabore o teste de todas as características de sua classe.
- Elabore o diagrama de classes deste sistema.

4) Você está desenvolvendo um simples jogo de cartas em Python. Crie duas classes: Carta e Baralho.

- Classe Carta:
  - Atributos: valor (2 a 10, J, Q, K, A) e naipe (Paus, Copas, Espadas, Ouros).
  - Método __init__(self, valor, naipe): O construtor da classe que inicializa os atributos.
- Classe Baralho:
  - Atributo: cartas, uma lista que armazena objetos do tipo Carta.
  - Métodos:

- O construtor da classe que inicializa o baralho com 52 cartas (de 2 a A para cada naipe).
- embaralhar(self): Um método que embaralha as cartas no baralho. (random)
- dar_carta(self): Um método que remove e retorna a carta do topo do baralho.

- Elabore o teste de todas as características de sua classe.

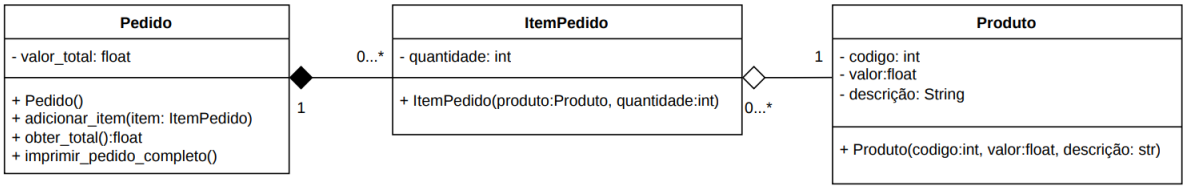
5) Desenvolva um sistema simples para gerenciar informações sobre alunos em uma escola. Crie duas classes: Aluno e Turma.

- Classe Aluno:
  - Atributos: nome, idade e nota (devem ser não publicos, crie gets e sets).
  - Método __init__(self, nome, idade, nota): O construtor da classe que inicializa os atributos.
- Classe Turma:
  - Atributo: alunos, uma lista que armazena objetos do tipo Aluno.
  - Métodos:
    - __init__(self): O construtor da classe que inicializa a lista de alunos.
    - adicionar_aluno(self, aluno): Um método que adiciona um aluno à turma.
    - remover_aluno(self, nome): Um método que remove um aluno com base no nome.
    - listar_alunos(self): Um método que exibe a lista de alunos na turma.
- Crie uma instância da classe Turma e teste os métodos de adição, remoção e listagem de alunos.
- Elabore o diagrama de classes deste sistema.

6) Você está desenvolvendo um sistema básico para gerenciar o estoque de produtos em um pequeno negócio. Crie duas classes: Produto e Estoque.

- Classe Produto:
  - Atributos: codigo, nome, preco e quantidade.
  - Método __init__(self, codigo, nome, preco, quantidade): O construtor da classe que inicializa os atributos.
- Classe Estoque:
  - Atributo: produtos, uma lista que armazena objetos do tipo Produto.
  - Métodos:
    - __init__(self): O construtor da classe que inicializa a lista de produtos.
    - adicionar_produto(self, produto): Um método que adiciona um produto ao estoque.
    - remover_produto(self, codigo): Um método que remove um produto do estoque com base no código.
    - listar_produtos(self): Um método que exibe a lista de produtos no estoque.
- Crie uma instância da classe Estoque e teste os métodos de adição, remoção e listagem de produtos.

7) Construa, teste e monte um menu iterativo para gerir um sistema de lojas com base no seguinte diagrama de classe:



Este é o diagrama mínimo. Você tem liberdade de acrescentar funcionalidades que achar pertinente.

8) Você vai criar um jogo de corrida simples utilizando o terminal, onde dois carros competem para alcançar a linha de chegada. O progresso dos carros será mostrado por meio de gráficos no terminal, representados por texto. Cada carro terá uma velocidade aleatória que é determinada a cada rodada e avançará na pista conforme essa velocidade. O primeiro carro a atingir a linha de chegada será o vencedor.

a) Criação de Carros:
   i) Crie uma classe Carro que tenha os seguintes atributos:
      - nome: o nome do carro (por exemplo, "Carro 1", "Carro 2").
      - posição: a posição atual do carro na pista (inicialmente 0).
      - velocidade: a velocidade do carro.

b) Comportamento do Jogo:
   i) A corrida ocorrerá em uma pista de comprimento fixo (por exemplo, 50 unidades).
   ii) A cada rodada, os carros aceleram e avançam em função da sua velocidade aleatoriamente .
   iii) O progresso dos carros será mostrado no terminal, utilizando o caractere “#” para representar a posição do carro na pista.
      Exemplo:
      Carro 1: ######
      Carro 2: ##########

c) A cada atualização do jogo, a posição dos carros deve ser atualizada e impressa novamente no terminal.

d) Vencedor: O jogo termina quando um dos carros atingir ou ultrapassar o final da pista (por exemplo, posição 50).

e) Quando um carro atinge a linha de chegada, imprima no terminal o nome do vencedor, por exemplo: